



United International University
Course: Operating Systems Laboratory
Assignment on Implementing System Call in xv6
Trimester: Fall 2025

Prepared by: Azizur Rahman Anik, Lecturer, dept. of CSE, UIU

Task : Implementing a System Call to observe the status of the last process that has completed execution.

1.1 Task Overview

Implement a new system call **proc_status** that will return the status (pid, turnaround time and the list of the system calls invoked by that process (at least once) tracked during its lifetime.

1.2 Requirements

1.2.1 System Call Implementation

After the return from your system call, you have a pointer (defined in user program) to a struct object. This struct object should contain:

1. The pid of the process last executed
2. The turnaround time (Completion time - Arrival time)
3. The number of times (at least once) a system call was called by that process

Add the following code in a new header file named pstat.h in the kernel folder and include it in files according to necessity.

```
#ifndef _PSTAT_H_
#define _PSTAT_H_
#include "syscall.h"
#define N_SYSCALLS 23
struct pstat {
    int pid;
    int turnaround;
    int syscall_count[N_SYSCALLS];
};
// Declaration only
extern struct pstat global_pstat;
#endif // _PSTAT_H_
// Global variable for process status
```

```
extern struct pstat global_pstat;
```

The signature of the system call is `int proc_status(uint64)` which needs to be added in `user/user.h` file.

1.2.2 User Program

You are given the user program `status.c`. Add this file into the user folder of the xv6 and add necessary codes in `Makefile` and `usys.pl` to boot this.

After completing the assignment, a sample output might look like this:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ grep hello README
$ status
PID: 3
Turnaround Time: 1
System Call Counts:
Syscall name read: 4
Syscall name exec: 1
Syscall name sbrk: 1
Syscall name open: 1
Syscall name close: 1
$ echo xv6
xv6
$ status
PID: 5
Turnaround Time: 0
System Call Counts:
Syscall name exec: 1
Syscall name sbrk: 1
Syscall name write: 2
$ status
PID: 6
Turnaround Time: 0
System Call Counts:
Syscall name exec: 1
Syscall name sbrk: 1
Syscall name write: 110
Syscall name proc_status: 1
```

1.2.3 Explanation

- grep command calls read system call 4 times, exec system call 1 time and so on. Turnaround time 1 means it takes 1 ticks to complete after being arrived in the system.

All information must be printed from user mode. Printing anything in kernel mode is not allowed.

1.2.4 Implementation Notes

- Please refer to the system call fstat to get an idea of how to return a structure object pointer.
- You need to modify syscall.c to count the occurrences of each system call
- Add necessary variables for turnaround time, number of times a process has called different system calls etc. in proc structure found in kernel/proc.h file.
- When a process is created, the kfork function in kernel/proc.c does the resource allocation and takes the process to Runnable state. This phase is the arrival time. Record the arrival time here with "ticks"
- When a process exits, the kexit function is invoked in kernel/proc.c. You may record the completion time here.
- You need to clear the variables during the resource deallocation phase as shown in the class
- Implement the system call in the sysproc.c file.
- You cannot modify pstat.h and status.c

Please DO NOT COPY solutions from anywhere (your friends, seniors, internet, etc.). Any form of plagiarism (irrespective of source or destination), will result in getting-100% marks in this assignment. You have to protect your code.

2.1 Submission Deadline

11:00 pm, Tuesday, 25 November 2025

3 Marks Distribution

SL	Sub Task	Marks
1	Counting System calls	3
2	Calculating Turnaround time	3
3	Argument Passing from kernel to user space	3
4	Properly recording arrival time, completion time and clearing values	3
5	Completion and showing outputs properly	3
	Total	15

Additionally there will be a viva during the evaluation and its marks will be adjusted to the marks of viva.